

Guía para crear un script capaz facilitar la creación de contenedores (postgres y dpage/pgadmin4)

Lo primero que haremos será tener una máquina virtual con sus instalaciones debidas de postgres, el servicio de Docker con sus respectivas imágenes (postgres y dpage/pgadmin4)

La finalidad de este script es no tener que estar escribiendo el comando completo, ya sea algunos de estos dos:

```
docker run --name contenedor-psql -e POSTGRES_PASSWORD=linux -d -p 2022:5432
postgres
```

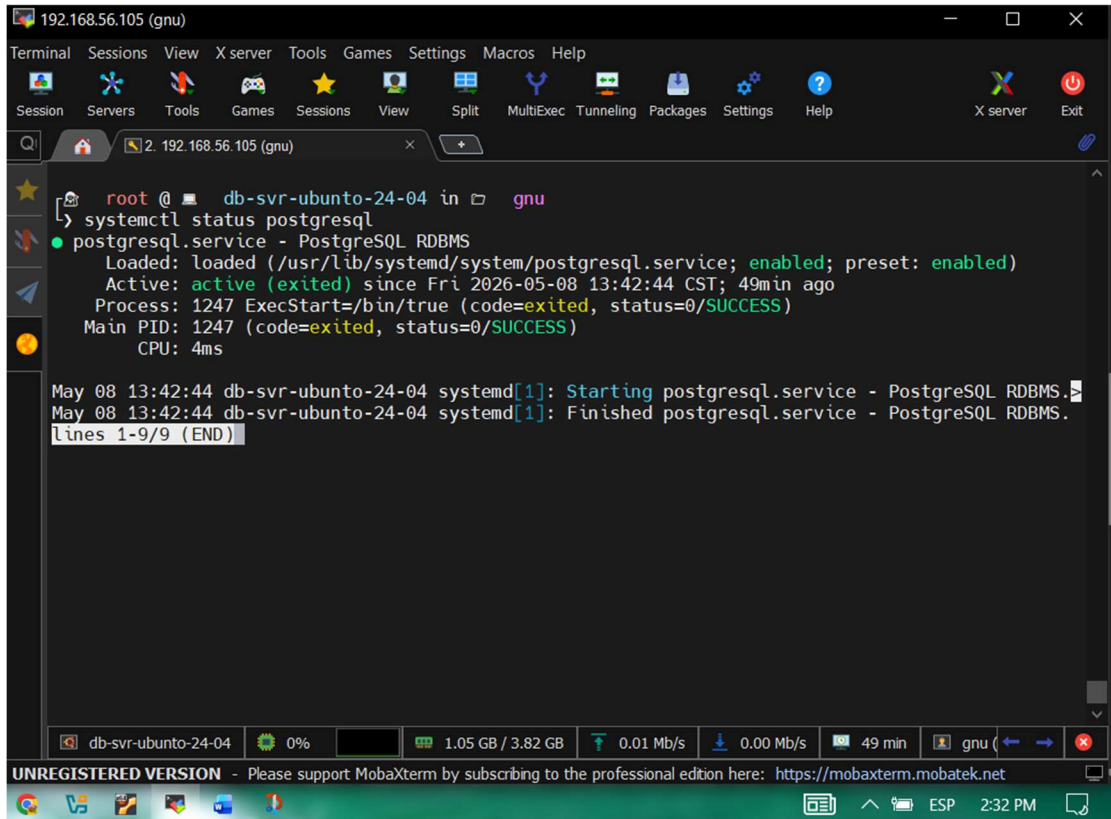
```
docker run -d -p 8080:80 --name nombrecontenedor -e
'PGADMIN_DEFAULT_EMAIL=user@domain.com' -e
'PGADMIN_DEFAULT_PASSWORD=linux' dpage/pgadmin4
```

Empezamos con la guía:

Primera parte: Verificación

Primero tenemos la maquina virtual con sus debidas instalaciones

Ejecutamos el siguiente comando para saber si el servicio está activo “systemctl status postgresql”



```
192.168.56.105 (gnu)
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help X server Exit

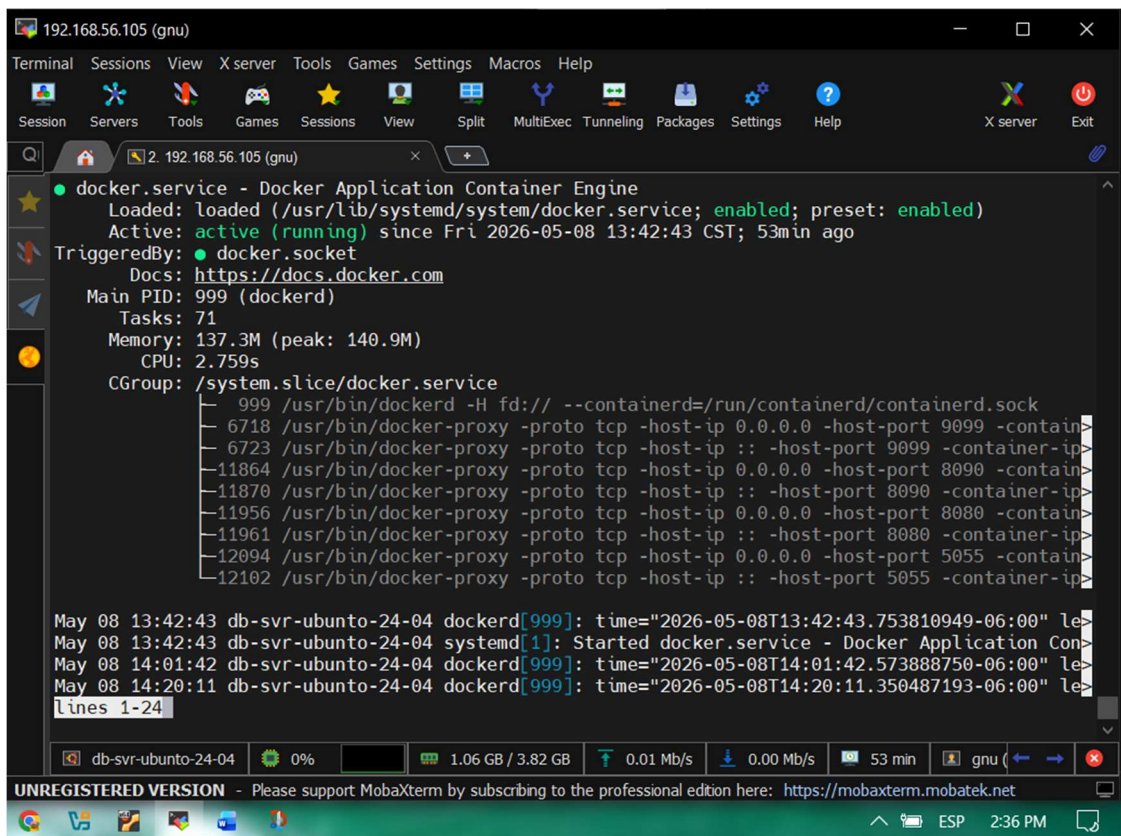
root @ db-svr-ubuntu-24-04 in gnu
systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Fri 2026-05-08 13:42:44 CST; 49min ago
   Process: 1247 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 1247 (code=exited, status=0/SUCCESS)
      CPU: 4ms

May 08 13:42:44 db-svr-ubuntu-24-04 systemd[1]: Starting postgresql.service - PostgreSQL RDBMS.
May 08 13:42:44 db-svr-ubuntu-24-04 systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
lines 1-9/9 (END)
```

db-svr-ubuntu-24-04 0% 1.05 GB / 3.82 GB 0.01 Mb/s 0.00 Mb/s 49 min gnu

UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

Ejecutamos el siguiente comando para saber si el servicio Docker está instalado “service docker status”



The screenshot shows a MobaXterm terminal window with the command 'service docker status' executed. The output indicates that the Docker service is loaded and active (running). It also shows the service's configuration, including its main PID (999), tasks (71), memory usage (137.3M), and CPU usage (2.759s). The service is managed by systemd and is part of the /system.slice/docker.service. The terminal also displays a list of Docker proxy processes and their network connections.

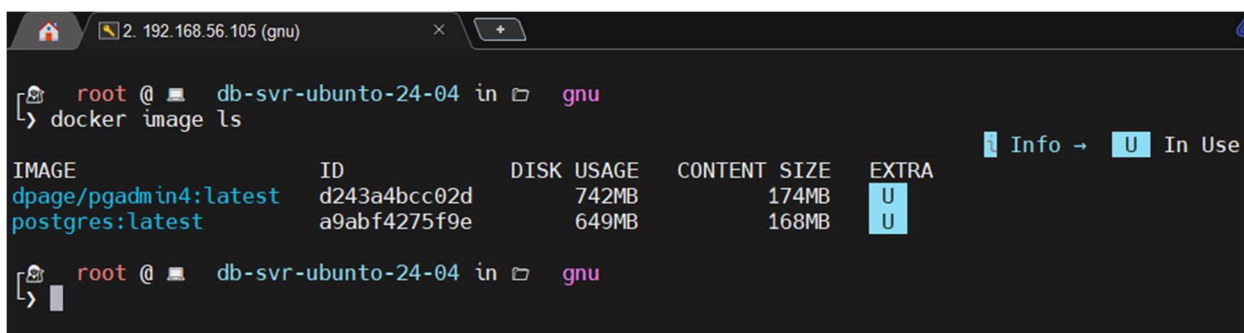
```
192.168.56.105 (gnu)
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help X server Exit

● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-05-08 13:42:43 CST; 53min ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 999 (dockerd)
      Tasks: 71
    Memory: 137.3M (peak: 140.9M)
       CPU: 2.759s
    CGroup: /system.slice/docker.service
            └─ 999 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
               6718 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9099 -container-
               6723 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 9099 -container-ip
               11864 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8090 -contain
               11870 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8090 -container-ip
               11956 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080 -contain
               11961 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8080 -container-ip
               12094 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 5055 -contain
               12102 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 5055 -container-ip

May 08 13:42:43 db-svr-ubuntu-24-04 dockerd[999]: time="2026-05-08T13:42:43.753810949-06:00" le
May 08 13:42:43 db-svr-ubuntu-24-04 systemd[1]: Started docker.service - Docker Application Con
May 08 14:01:42 db-svr-ubuntu-24-04 dockerd[999]: time="2026-05-08T14:01:42.573888750-06:00" le
May 08 14:20:11 db-svr-ubuntu-24-04 dockerd[999]: time="2026-05-08T14:20:11.350487193-06:00" le
lines 1-24
db-svr-ubuntu-24-04 0% 1.06 GB / 3.82 GB 0.01 Mb/s 0.00 Mb/s 53 min gnu
UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net
ESP 2:36 PM
```

También con este vemos si sus respectivas imágenes están descargadas “doceker image ls”

Debemos tener esas 2 que son las necesarias



The screenshot shows a MobaXterm terminal window with the command 'docker image ls' executed. The output displays a table of Docker images, including their names, IDs, disk usage, and content size. The images listed are 'dpage/pgadmin4:latest' and 'postgres:latest'. The terminal also shows the user's current directory and the command prompt.

```
root @ db-svr-ubuntu-24-04 in gnu
[> docker image ls

IMAGE                ID                DISK  USAGE  CONTENT  SIZE  EXTRA
dpage/pgadmin4:latest d243a4bcc02d      742MB 174MB
postgres:latest      a9abf4275f9e      649MB 168MB

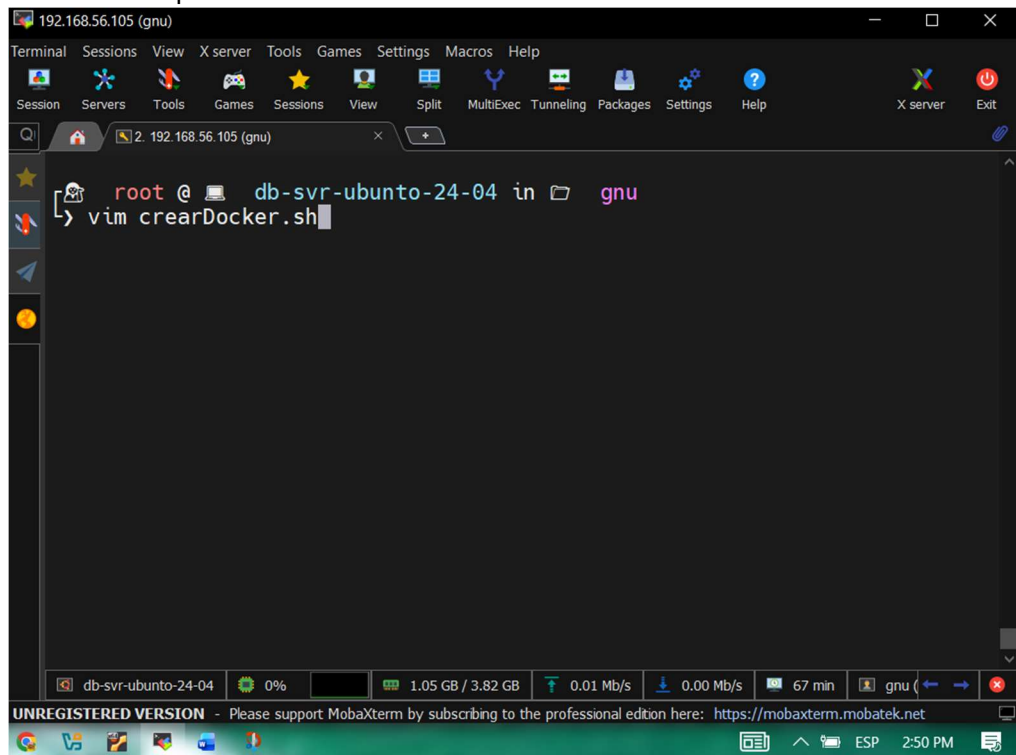
root @ db-svr-ubuntu-24-04 in gnu
[>
```

Lo anteriormente mostrado simplemente fue para verificar las herramientas que debemos tener instaladas

Segunda parte: creación del script

Primero crearemos el scrip con “vim crearDocker.sh”

En este caso puede ser vim o nano



No se preocupen por la calidad al final mandare el código listo para copiar y pegar
Ya dentro solamente copiamos lo siguiente y guardamos/salimos con “:wq”

```
#!/bin/bash

# Limpiar la pantalla
clear

echo "=====
echo "          Menú de Docker - Bases de Datos          "
echo "=====
echo " 1) Docker Web (Solo PgAdmin4)"
echo " 2) Docker por la App (Solo PostgreSQL)"
echo " 3) Instalar Ambos (PostgreSQL + PgAdmin4)"
echo " 4) Salir del menú"
echo "=====
-p "Elige una opción (1-4): " OPCION
read OPCION
echo ""

case $OPCION in
1)
    echo "---[ Configurando Docker Web (PgAdmin4) ]---"
    read -p "Nombre del contenedor (ej. mjpgadmin): " NOMBRE
    read -p "Puerto local (ej. 8080): " PUERTO
    read -p "Correo administrador: " EMAIL
    read -s -p "Contraseña: " PASSWORD
    echo ""

    docker run -d -p $(PUERTO):80 --name $(NOMBRE) \
    -e "PGADMIN_DEFAULT_EMAIL=$(EMAIL)" \
    -e "PGADMIN_DEFAULT_PASSWORD=$(PASSWORD)" \
    dpage/pgadmin4

    echo ""
    echo "[+] ¡Listo! PgAdmin web disponible en: http://localhost:$(PUERTO)"
    ;;

2)
    echo "---[ Configurando Docker por la App (PostgreSQL) ]---"
    read -p "Nombre del contenedor (ej. mpostgres): " NOMBRE
    read -p "Puerto local (ej. 5432): " PUERTO
    read -s -p "Contraseña de Postgres: " PASSWORD
    echo ""

    docker run --name $(NOMBRE) \
    -e POSTGRES_PASSWORD=$(PASSWORD) \
    -d -p $(PUERTO):5432 postgres

    echo ""
    echo "[+] ¡Listo! PostgreSQL app disponible en el puerto: $(PUERTO)"
    ;;

3)
    echo "---[ Configurando Ambos Servicios ]---"
    read -p "Nombre Base para los contenedores (ej. m1db): " NOMBRE
    read -p "Puerto para PostgreSQL (ej. 5432): " PUERTO_PGSQL
    read -p "Puerto para PgAdmin (ej. 8080): " PUERTO_PGADMIN
    read -p "Correo administrador PgAdmin: " EMAIL
    read -s -p "Contraseña (se usará para ambos): " PASSWORD
    echo ""

    # Crear PostgreSQL
    docker run --name "${NOMBRE}-psql" -e POSTGRES_PASSWORD=$(PASSWORD) -d -p $(PUERTO_PGSQL):5432 postgres

    # Crear PgAdmin
    docker run --name "${NOMBRE}-pgadmin" -e "PGADMIN_DEFAULT_EMAIL=$(EMAIL)" -e "PGADMIN_DEFAULT_PASSWORD=$(PASSWORD)" -d -p $(PUERTO_PGADMIN):80 dpage/pgadmin4

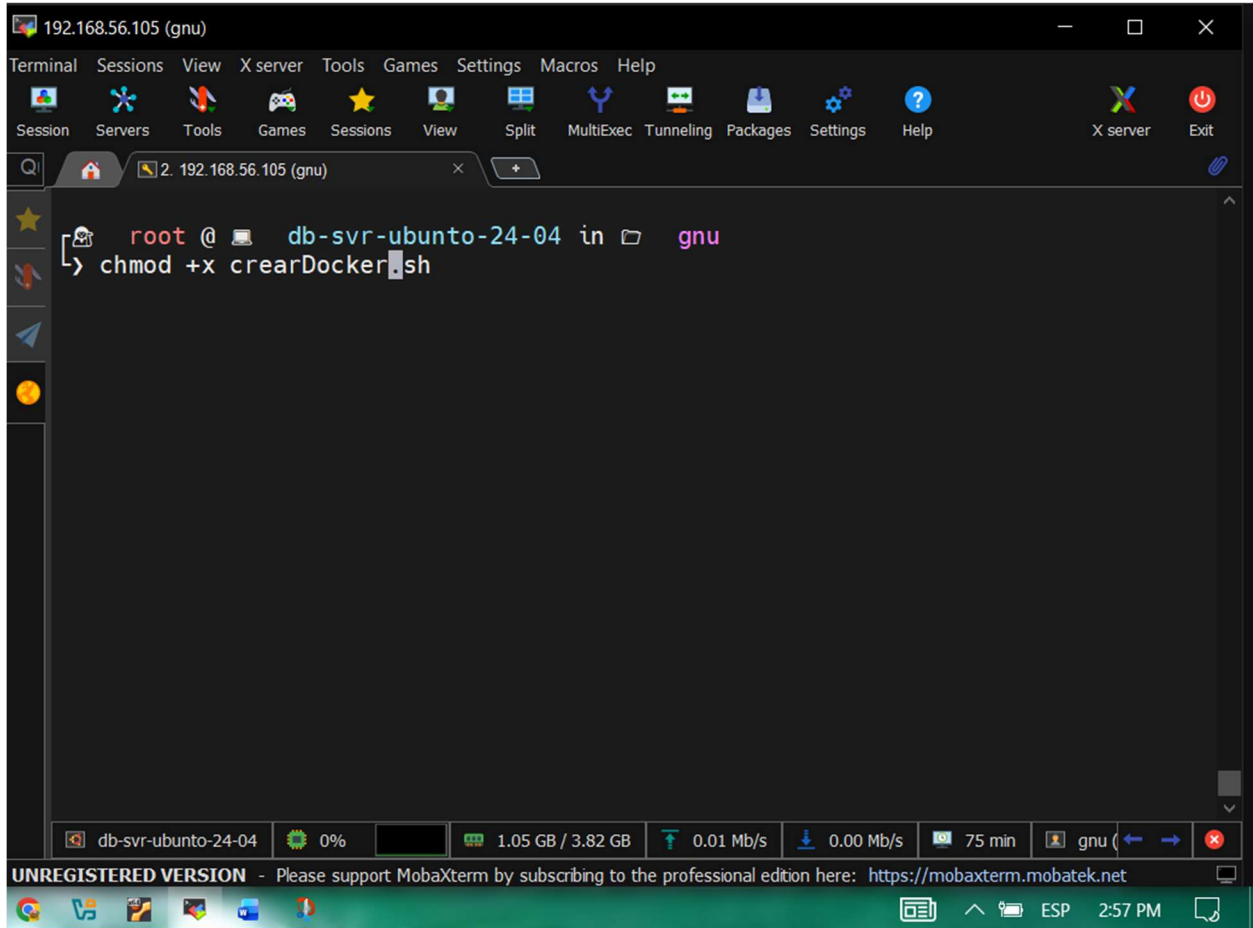
    echo ""
    echo "[+] ¡Listo! Ambos contenedores han sido creados."
    echo " :Base de datos: puerto $(PUERTO_PGSQL)"
    echo " :Web PgAdmin: http://localhost:$(PUERTO_PGADMIN)"
    ;;

4)
    echo "Saliendo sin hacer cambios..."
    exit 0
    ;;

*)
    echo "Opción no válida. Por favor ejecute el script nuevamente y elige 1, 2, 3 o 4."
    ;;
esac
echo "=====
```

Luego de guardar y salir ejecutamos el siguiente comando para darle permisos de ejecución

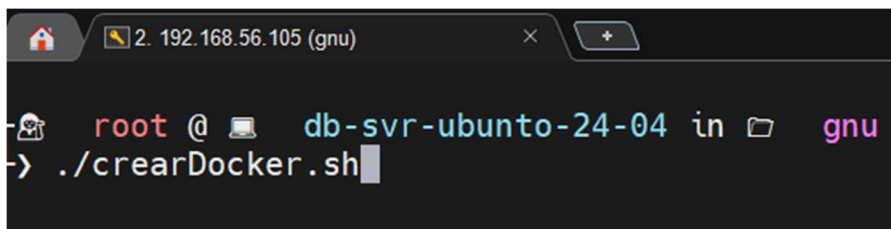
“chmod +x crearDocker.sh”



The screenshot shows a MobaXterm window titled "192.168.56.105 (gnu)". The terminal interface displays the prompt "root @ db-svr-ubuntu-24-04 in gnu" followed by the command "chmod +x crearDocker.sh". The command has been executed successfully. The window includes a menu bar with options like Terminal, Sessions, View, X server, Tools, Games, Settings, Macros, and Help. A toolbar with various icons is visible below the menu. The status bar at the bottom shows system information and a notice about the unregistered version of MobaXterm.

```
root @ db-svr-ubuntu-24-04 in gnu
-> chmod +x crearDocker.sh
```

Luego solo ejecutamos el script con “./crearDocker.sh”



The screenshot shows a MobaXterm window titled "2. 192.168.56.105 (gnu)". The terminal interface displays the prompt "root @ db-svr-ubuntu-24-04 in gnu" followed by the command "./crearDocker.sh". The command has been executed successfully.

```
root @ db-svr-ubuntu-24-04 in gnu
-> ./crearDocker.sh
```


Su ejecución seria la siguiente:

```
=====
                          Menú de Docker - Bases de Datos
=====
1) Docker Web (Solo PgAdmin4)
2) Docker por la App (Solo PostgreSQL)
3) Instalar Ambos (PostgreSQL + PgAdmin4)
4) Salir del menú
=====
Eliga una opción (1-4): 2

---[ Configurando Docker por la App (PostgreSQL) ]---
Nombre del contenedor (ej. mipostgres): PruebaScript
Puerto local (ej. 2022): 7077
Contraseña de Postgres:
0c70d332659deae7228bad1c6c3514efdb69743d259f516320321a0b1d7c3eb6

[+] ¡Listo! PostgreSQL app disponible en el puerto: 7077
=====

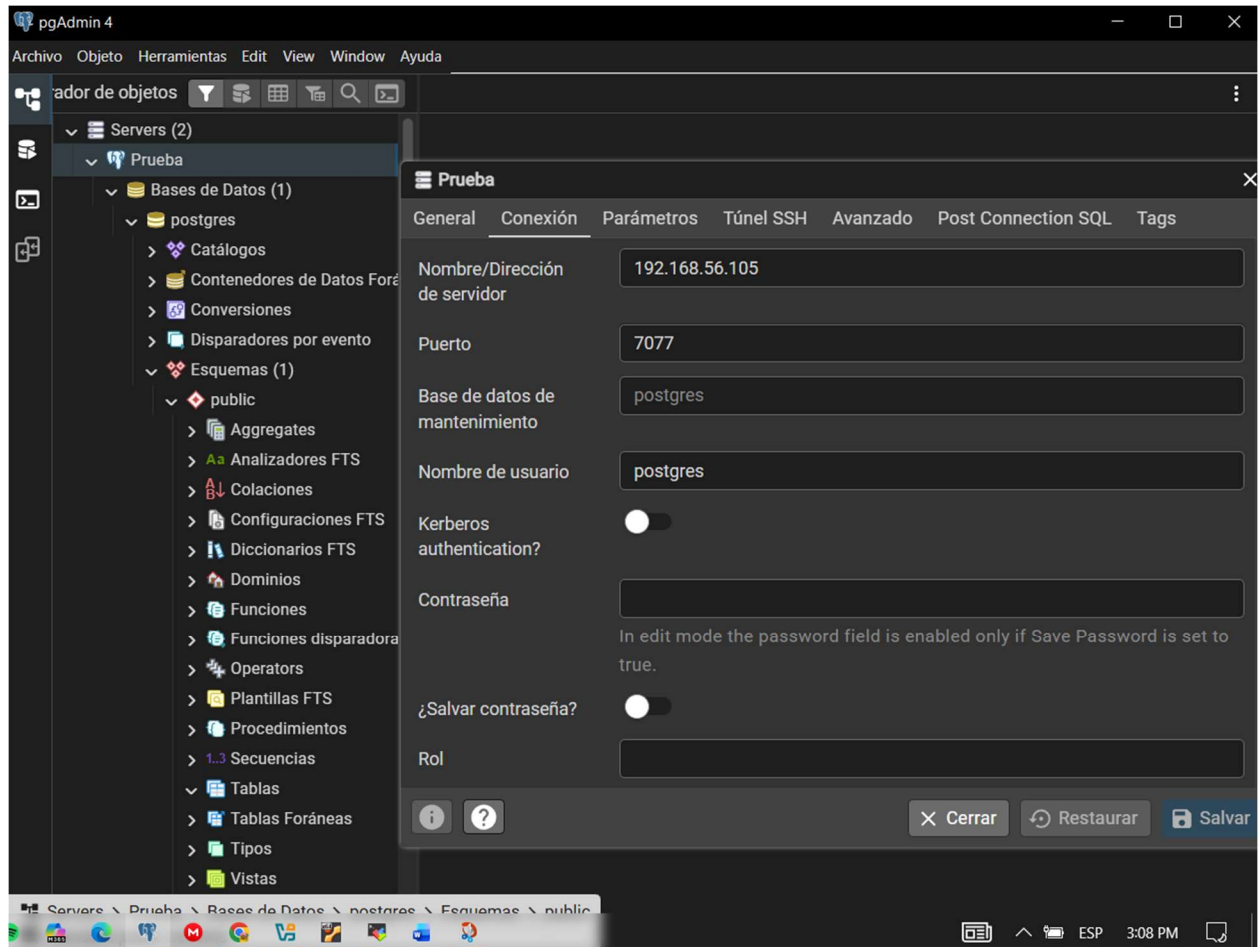
root @ db-svr-ubuntu-24-04 in gnu
```

Y verificamos su creación con “docker ps” o “docker ps -a” porque puede ser que este creado pero apagado

```
root @ db-svr-ubuntu-24-04 in gnu
-> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
0c70d332659d   postgres  "docker-entrypoint.s..." 2 minutes ago Up 40 seconds 0.0.0.0:7077->5432/tcp, :::7077->5432/tcp PruebaScript

root @ db-svr-ubuntu-24-04 in gnu
```

Verificación de pgadmin



Ahora la prueba con pgadmin web

```
=====
                          Menú de Docker - Bases de Datos
=====
1) Docker Web (Solo PgAdmin4)
2) Docker por la App (Solo PostgreSQL)
3) Instalar Ambos (PostgreSQL + PgAdmin4)
4) Salir del menú
=====
Elija una opción (1-4): 1

---[ Configurando Docker Web (PgAdmin4) ]---
Nombre del contenedor (ej. mipgadmin): PruebaWeb
Puerto local (ej. 8080): 9999
Correo administrador: yan@gmail.com
Contraseña:
a0f5ed960250b23353faa82c717645f049ae77a6601ba0e75cda1cfb5388391c

[+] ¡Listo! PgAdmin web disponible en: http://localhost:9999
=====

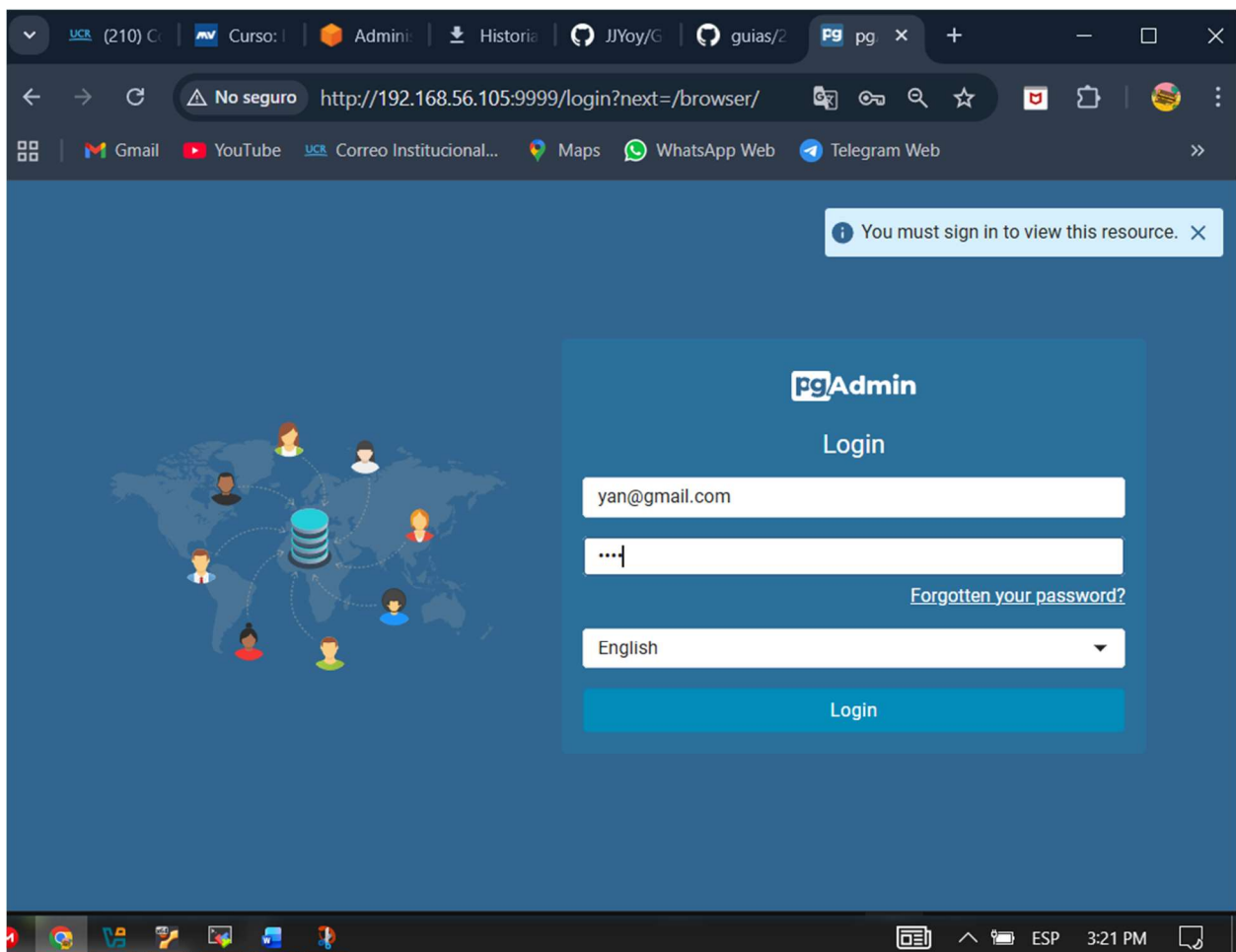
[🏠] root @ 🖥 db-svr-ubuntu-24-04 in 📁 gnu
```

Verificamos

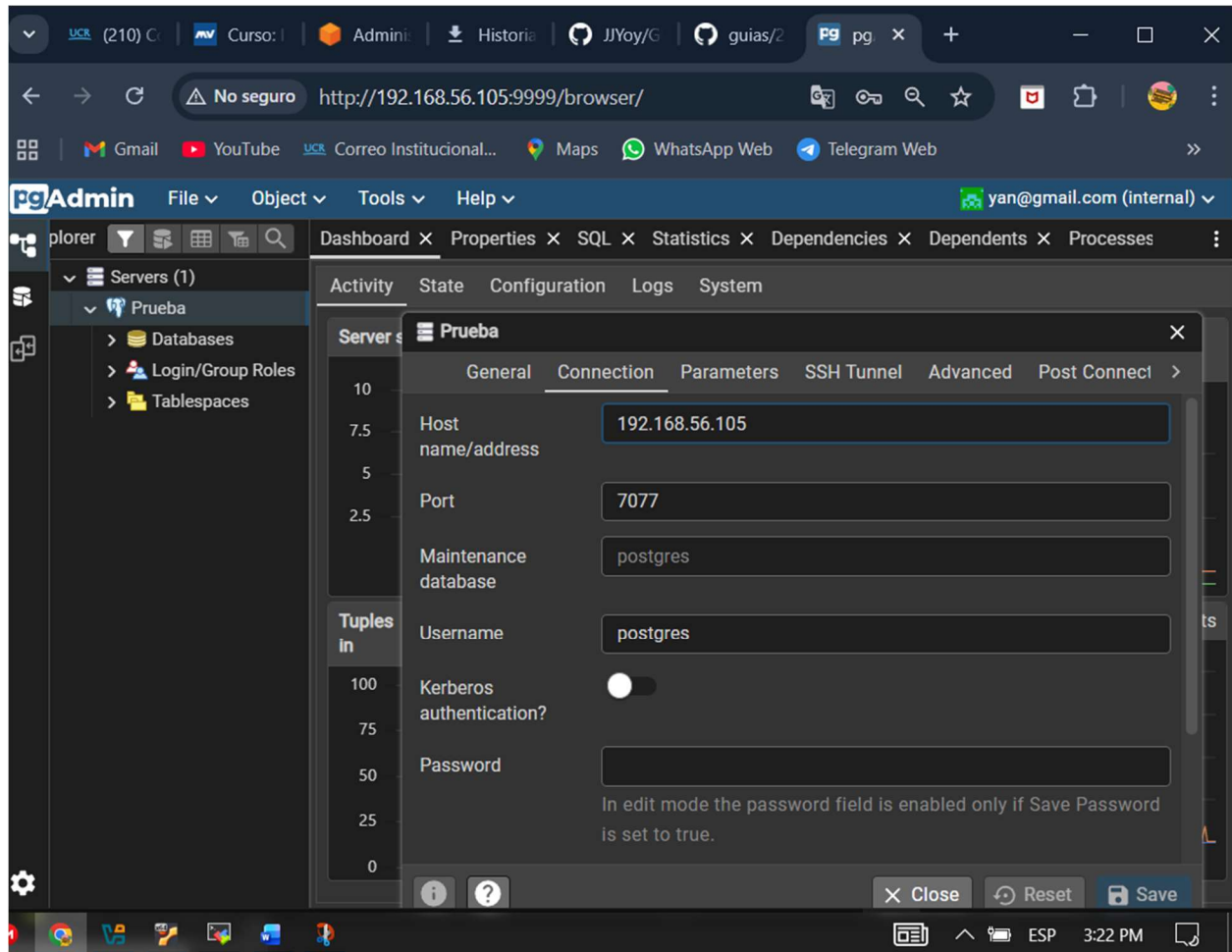
```
root@db-svr-ubuntu-24-04:~# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
a0f5ed966250  dpag/pgadmin4  "/entrypoint.sh"        About a minute ago    Up About a minute    443/tcp, 0.0.0.0:9999->80/tcp, [::]:9999->80/tcp    PruebaWeb
0c70d332659d  postgres      "docker-entrypoint.s..."  12 minutes ago       Up 11 minutes        0.0.0.0:7077->5432/tcp, [::]:7077->5432/tcp        PruebaScript
```

db-svr-ubuntu-24-04 53% 0.68 GB / 3.82 GB 0.01 Mb/s 0.00 Mb/s 93 min gnu (x2) /: 1% /usr: 31% /BD_Produccion: 1% /BD_Desarrollo: 1% /boot: 3% /home: 1% /tmp: 1% /var: 30%

Buscar



Y con esto verificamos el contenedor desde el pgadmin web



Este sería el script:

```
#!/bin/bash

# Limpiar la pantalla
clear

echo "=====
echo "      Menú de Docker - Bases de Datos      "
echo "=====
echo " 1) Docker Web (Solo PgAdmin4)"
echo " 2) Docker por la App (Solo PostgreSQL)"
echo " 3) Instalar Ambos (PostgreSQL + PgAdmin4)"
echo " 4) Salir del menú"
echo "=====
read -p "Eliga una opción (1-4): " OPCION
echo ""

case $OPCION in
1)
    echo "---[ Configurando Docker Web (PgAdmin4) ]---"
    read -p "Nombre del contenedor (ej. mipgadmin): " NOMBRE
    read -p "Puerto local (ej. 8080): " PUERTO
    read -p "Correo administrador: " EMAIL
    read -s -p "Contraseña: " PASSWORD
    echo ""

    docker run -d -p ${PUERTO}:80 --name ${NOMBRE} \
    -e "PGADMIN_DEFAULT_EMAIL=${EMAIL}" \
    -e "PGADMIN_DEFAULT_PASSWORD=${PASSWORD}" \
    dpage/pgadmin4
```

```
echo ""  
echo "[+] ¡Listo! PgAdmin web disponible en: http://localhost:${PUERTO}"  
;;
```

2)

```
echo "---[ Configurando Docker por la App (PostgreSQL) ]---"  
read -p "Nombre del contenedor (ej. mipostgres): " NOMBRE  
read -p "Puerto local (ej. 2022): " PUERTO  
read -s -p "Contraseña de Postgres: " PASSWORD  
echo ""
```

```
docker run --name ${NOMBRE} \  
-e POSTGRES_PASSWORD="${PASSWORD}" \  
-d -p ${PUERTO}:5432 postgres
```

```
echo ""  
echo "[+] ¡Listo! PostgreSQL app disponible en el puerto: ${PUERTO}"  
;;
```

3)

```
echo "---[ Configurando Ambos Servicios ]---"  
read -p "Nombre base para los contenedores (ej. mibd): " NOMBRE  
read -p "Puerto para PostgreSQL (ej. 2022): " PUERTO_PSQL  
read -p "Puerto para PgAdmin (ej. 8080): " PUERTO_PGADMIN  
read -p "Correo administrador PgAdmin: " EMAIL  
read -s -p "Contraseña (se usará para ambos): " PASSWORD  
echo ""
```

```
# Crear PostgreSQL
```

```
docker run --name "${NOMBRE}-psql" -e POSTGRES_PASSWORD="${PASSWORD}" -d -p
${PUERTO_PSQL}:5432 postgres
```

```
# Crear PgAdmin
```

```
docker run --name "${NOMBRE}-pgadmin" -e "PGADMIN_DEFAULT_EMAIL=${EMAIL}" -e
"PGADMIN_DEFAULT_PASSWORD=${PASSWORD}" -d -p ${PUERTO_PGADMIN}:80
dpage/pgadmin4
```

```
echo ""
```

```
echo "[+] ¡Listo! Ambos contenedores han sido creados."
```

```
echo " :Base de datos: puerto ${PUERTO_PSQL}"
```

```
echo " :Web PgAdmin: http://localhost:${PUERTO_PGADMIN}"
```

```
::
```

```
4)
```

```
echo "Saliendo sin hacer cambios..."
```

```
exit 0
```

```
::
```

```
*)
```

```
echo "Opción no válida. Por favor ejecuta el script nuevamente y elige 1, 2, 3 o 4."
```

```
::
```

```
esac
```

```
echo "=====
```